COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Module 06: Discrete mathematical reasoning

Expanded Volume 02 / individual teaching module

Four lessons with derivations, runnable examples, practical work and 20 quiz questions.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Module 06 — Discrete mathematical reasoning

Original prerequisites: Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

Original module project: Write a mathematical notebook containing a truth-table specification, an induction proof, a graph model and a justified operation-count estimate. Include counterexamples to three false claims.

Expanded project: A mathematical specification notebook. A versioned document plus scripts containing five quantified world rules, complete door truth tables, two proofs, a six-node dependency graph, two recurrence analyses, and three counterexamples.

Acceptance: The proof reviewer can identify initialization/preservation/termination where applicable. Every graph edge has a stated meaning; cyclic input is rejected. The ten algebra-bridge items and all 15 module problems have independent attempts and corrections.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 06.1 — Algebra, functions, logarithms and trigonometry

Learning outcome: Translate a game relationship into an equation with a defined domain.

Start with a relationship, not a remembered formula

A useful mathematical model identifies quantities, their units, and the inputs for which its rules make sense. Suppose a unit moves a straight-line distance d at a constant positive speed v. Its travel time is t = d/v. This model deliberately omits obstacles, acceleration, pauses, and turning. A route planner may later supply a path length, but a straight-line distance is not automatically that path length.

An equation describes a relationship; an assignment performs an update. In mathematics, d = vt can be rearranged to v = d/t only when t is nonzero. A simulation assignment such as `distance = speed * time` does not make future changes to speed update the stored distance. Preserve the distinction learned in Volume 1.

Rearrange while preserving the domain

To solve 3t + 2 = 14, subtract 2 from both sides and then divide both sides by 3. Each operation preserves equality, yielding t = 4. Division by an expression is more delicate: cancelling x from x(x - 2) = 0 would discard the valid solution x = 0. Factoring instead gives x = 0 or x = 2. Squaring both sides can introduce extra solutions, so substitute candidates into the original equation.

For a real function sqrt(x - 4), the domain is x >= 4. For 1/(x - 4), it is x != 4. For log2(x - 4), it is x > 4. A domain is part of a function's contract. Rejecting impossible inputs clearly is more useful than allowing a calculation to produce a plausible-looking but meaningless result.

Exponents and logarithms describe scale

The identity 2^k = n is equivalent to k = log2(n) for n > 0. Repeatedly halving 1024 reaches 1 after ten divisions. For 1000, log2(1000) is not an integer. An algorithm that floors each half, and one that rounds each half up, can require different integer step counts. State the exact recurrence before turning a logarithm into a loop bound.

A logarithm needs a dimensionless argument. Writing log(seconds) without a reference scale is incomplete; log(t/t0) is meaningful when t and t0 use the same units. This habit becomes important when fitting curves or comparing quantities that use different measurement scales.

Angles and the unit circle

A radian measures arc length divided by circle radius. One revolution is 2*pi radians, so degrees convert by theta_rad = theta_deg*pi/180. In an x-right, y-up plane, the unit direction at angle theta measured counterclockwise from +x is (cos(theta), sin(theta)). A speed v then produces velocity (v*cos(theta), v*sin(theta)). Its displacement over dt is that velocity multiplied by dt.

The unit-circle identity cos(theta)^2 + sin(theta)^2 = 1 explains why the direction has length one. Multiplying its components by v scales the speed without changing its angle. Python's trigonometric functions use radians; displaying an angle in degrees does not change the function's input contract.

Exact mathematical zeros can appear as tiny floating-point values, which is a representation issue rather than a new geometric direction. [R04, R40]

Worked derivation

A unit travels 36 m at 6 m/s. The model requires d >= 0 and v > 0, so t = 36/6 = 6 s. At a heading of 90 degrees, theta = pi/2, giving velocity (0, 6) m/s in the stated y-up coordinates. During dt = 0.25 s, displacement is (0, 1.5) m.

Check the units: (m/s)*s = m. Check the magnitude: sqrt(0^2 + 6^2) = 6 m/s. Check the domain: speed zero does not produce a finite arrival time for a positive distance. It should be reported as an invalid input or unreachable state, not changed to an arbitrary small number.

For a rendering budget f = 120 frames/s, the interval is 1000/f = 8.333... ms/frame. This is an arithmetic target, not evidence that a machine can meet it.

Runnable demonstration

Run `python run_example.py 06.1` from the extracted package folder. Predict before executing.

```
import math
from mathkit import travel_time
print(f"travel time: {travel_time(36, 6):.1f} s")
angle = math.radians(90)
print(f"direction: ({math.cos(angle):.3f}, {math.sin(angle):.3f})")
print(f"halvings of 1024: {int(math.log2(1024))}")
```

Recorded expected output:

```
travel time: 6.0 s
direction: (0.000, 1.000)
halvings of 1024: 10
```

Practical assignment

Original brief: Solve distance = speed*time for each variable and state forbidden inputs. Convert 0, 90, 180 and 360 degrees to radians. Pass: solve ten algebra problems and explain logarithms using repeated halving before advancing.

1. Implement travel_time(distance, speed), requiring finite nonnegative distance and finite positive speed. Return seconds for inputs in meters and meters/second.
2. Complete the original ten algebra problems; the readiness bridge supplies concrete practice. Show all rearrangements and domain restrictions.
3. Tabulate directions at 0, 90, 180 and 360 degrees. Explain near-zero floating-point components rather than forcing all small numbers to zero.
4. Compare floor-halving and ceiling-halving for sizes 1, 2, 3, 1000 and 1024. Record the chosen recurrence.

Coding slice: Work in `labs/student/lab_06_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not pass the number 90 directly to a radian-based sine function, cancel a possibly zero factor, or round a logarithm before defining the discrete problem.

Independent transfer task

A unit moves at 8 m/s at 30 degrees for 0.5 s. Find its displacement in the stated y-up plane.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is log2(64)?
2. Convert 180 degrees to radians.
3. What restriction applies when computing speed = distance/time?
4. State the real domain of f(x) = sqrt(x-2)/(x-5).
5. Solve x(x-2)=0 without losing a solution.

Assigned study

R40: functions and domains, exponential/logarithmic functions, and the unit circle. R08: prerequisite review. R06: sums and asymptotic growth. Use the source topics, not guessed exercise numbers.

Original reading assignment retained: R08: prerequisite algebra review and trigonometric functions; R06: sums and asymptotics.

Lesson 06.2 — Logic, sets and quantified claims

Learning outcome: Express requirements precisely enough to test or prove.

Convert a requirement into a claim

A requirement such as "all living units have transforms" contains a domain and a predicate. Let U be the set of units and let live(u) and has_transform(u) be predicates. The precise claim is: for every u in U, live(u) implies has_transform(u). It does not say dead units lack transforms, or that transformed units must be living.

An implication P -> Q is equivalent to (not P) or Q. Its only false case is P true and Q false. When P is false, the implication places no obligation on Q. This is not a causal assertion. It is a condition that rules out one combination of truth values.

Quantifiers have different obligations

A universal claim requires every allowed case; an existential claim requires a witness. To disprove "every saved entity has a unique ID," one repeated ID is enough. To establish "some unit can reach the exit," provide one allowed unit and an actual route or a justified reachability argument.

Order matters. "Every player can choose some spawn point" allows different spawn points for different players. "There is a spawn point every player can choose" requires one common point. A two-player map with disjoint allowed spawn sets satisfies the first claim but not the second. A test generator must reflect the correct quantifier order.

Negation changes the quantifier

The negation of "for every u, P(u)" is "there exists u for which not P(u)." The negation of "there exists u for which P(u)" is "for every u, not P(u)." Negating only the predicate while keeping the quantifier is a frequent specification error.

For a universal statement over an empty set, there is no counterexample, so the statement is true. For an existential statement over that set, there is no witness, so it is false. Therefore a validator that checks every entity's ID can return true for a file with zero entities. A separate nonempty-world requirement is needed when emptiness is forbidden.

Sets, relations, and functions

A set contains distinct elements; a list can repeat elements and preserve an order. The set {2, 2, 5} has two elements, while the list [2, 2, 5] has three entries. A relation is a set of ordered pairs. A function is a relation assigning exactly one output to each input in its declared domain. A unit-to-transform mapping must therefore neither omit required units nor give a unit two incompatible authoritative transforms.

Conjunction means both conditions must hold; disjunction means at least one holds unless "exclusive or" is explicitly stated. For the door rule key OR (level >= 5 AND stamina >= 10), a key suffices regardless of the other two predicates. Parentheses communicate the intended grouping. Build the eight-row Boolean truth table first, then separately test the numeric comparisons at 4/5/6 and 9/10/11. The truth table does not replace boundary tests of how those predicates are computed. [R06]

Worked derivation

Suppose U = {A, B, C}, with health values 5, 0, and 8. The claim "every unit has positive health" is false because B supplies a counterexample. The claim "some unit has positive health" is true; either A or C is a witness.

For the guarded requirement live(u) -> has_transform(u), a dead B with no transform does not violate the requirement. A living A without a transform does. To require transforms for all units, remove the live guard rather than trying to repair the truth table.

Negate "every active unit has a valid owner": there exists an active unit whose owner is invalid. It is not "every active unit has an invalid owner."

Runnable demonstration

Run `python run_example.py 06.2` from the extracted package folder. Predict before executing.

```
from mathkit import implies
for p in (False, True):
    for q in (False, True):
        print(f"{int(p)} -> {int(q)} = {int(implies(p, q))}")
health = [5, 0, 8]
print("all positive:", all(h > 0 for h in health))
print("counterexample indices:", [i for i, h in enumerate(health) if h <= 0])
```

Recorded expected output:

```
0 -> 0 = 1
0 -> 1 = 1
1 -> 0 = 0
1 -> 1 = 1
all positive: False
counterexample indices: [1]
```

Practical assignment

Original brief: Formalize five requirements for the grid game using sets and quantifiers. Construct truth tables for a door-access rule. Pass: provide a counterexample or witness for each negated requirement.

1. Implement implies(p, q) for actual Boolean inputs, rejecting integer lookalikes under this exercise contract.
2. Write five quantified requirements for Training Yard or a small fictional unit world. State each domain explicitly.
3. Construct the eight-row key/level/stamina predicate table, then test the numeric boundaries independently.
4. For each requirement, provide a witness or counterexample for its negation. Include an empty collection and a repeated ID.

Coding slice: Work in `labs/student/lab_06_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Implication is not its converse. A validator passing a universal check on an empty collection does not establish that the collection is nonempty.

Independent transfer task

Two players have allowed spawn sets {A} and {B}. Which quantifier order is satisfied?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. When is P -> Q false?
2. Negate "every unit has positive health."
3. Why must the domain of a quantified statement be explicit?
4. What are the truth values of "all elements are positive" and "some element is positive" on an empty set?
5. Does P -> Q imply Q -> P? Give a game counterexample.

Assigned study

R06: propositions, predicates, sets, relations, and quantifier order. Complete a proof or truth-table exercise for each form used in the lab.

Original reading assignment retained: R06: propositions, predicates, sets and relations.

Lesson 06.3 — Proofs, induction and invariants

Learning outcome: Justify a program property instead of testing only examples.

A proof connects assumptions to a conclusion

Testing evaluates selected executions. A proof explains why a claim follows for every case within a mathematical model. Neither substitutes completely for the other: the proof may model integer arithmetic while the implementation uses finite-width numbers, or the implementation may not match the proven algorithm.

A direct proof starts with the premises and uses justified steps. To show an even integer plus an even integer is even, write them as 2a and 2b. Their sum is 2(a+b), which has the defining form of an even integer. A proof by contradiction assumes the negation of the desired result and derives an impossibility under the stated premises.

Induction is a controlled chain

For a claim P(n) over nonnegative integers, establish P(0). Then assume P(k) for an arbitrary allowed k and derive P(k+1). The inductive hypothesis is a temporary premise used to prove the step; it is not permission to assume the conclusion without connecting the algebra.

For S(n) = 1 + ... + n, the base S(0) = 0 matches 0(1)/2. Assuming S(k) = k(k+1)/2, add k+1 to obtain (k+1)(k+2)/2. This proves the formula at the next integer. Checking n=1, 2 and 3 alone does not establish the chain.

Put an invariant at a precise program boundary

To count living units, let i be the number already inspected and c the count of positive-health values among indices 0 through i-1. At the top of the loop, require 0 <= i <= N and c equals exactly that prefix count. Before the loop, i=0 and c=0, so the empty prefix establishes the invariant.

During an iteration, inspect the element at i. Increment c exactly when its health is positive, then increment i. The newly processed prefix has the correct count, preserving the invariant. At exit i=N, the prefix is the whole collection, so c is the desired total.

Termination is a separate obligation

The quantity N-i is a nonnegative integer that decreases by one per iteration. It cannot decrease forever while remaining nonnegative; this supplies a termination argument. An invariant by itself does not guarantee termination. A loop that never changes i might preserve a true statement and still run forever.

Separate partial correctness (the result is correct if the loop terminates) from total correctness (it also terminates under the premises). Record invalid-input behavior separately: if k exceeds the collection length, this exercise rejects the call instead of redefining the claimed prefix.

Connect the model back to code

The implementation reads finite health values and counts strictly greater than zero. A different rule, such as health >= 0, requires a different predicate and different boundary tests. Mathematical integers do not overflow; later native-code lessons will add range constraints. Code review should check initialization, the comparison, the update order, and the exit condition against the proof, not merely verify that the comments sound mathematical. [R06]

Worked derivation

Take health = [4, 0, -1, 9]. The prefix counts for k=0,1,2,3,4 are 0,1,1,1,2. At k=3, the invariant says c=1 because only the first entry among [4,0,-1] is positive. The next step reads 9, increments c to 2, and extends the prefix to four entries.

For the sum formula at n=100, n(n+1)/2 = 5050. The demonstration compares this with a loop as an implementation check. The induction proof, rather than those two matching numbers, justifies the general formula.

Runnable demonstration

Run `python run_example.py 06.3` from the extracted package folder. Predict before executing.

```
from mathkit import count_positive_prefix
health = [4, 0, -1, 9]
for k in range(len(health) + 1):
    print(f"prefix {k}: {count_positive_prefix(health, k)} living")
n = 100
print("sum and formula:", sum(range(n + 1)), n * (n + 1) // 2)
```

Recorded expected output:

```
prefix 0: 0 living
prefix 1: 1 living
prefix 2: 1 living
prefix 3: 1 living
prefix 4: 2 living
sum and formula: 5050 5050
```

Practical assignment

Original brief: Prove the formula 1+...+n = n(n+1)/2 by induction. Then write an invariant for counting living units in a prefix of a list. Pass: show initialization, preservation and termination explicitly.

1. Implement count_positive_prefix(values, k); allow an empty list with k=0, reject nonfinite data and invalid k, and do not modify inputs.
2. Write the full induction proof for 1+...+n, including n=0 and the algebraic step.
3. Write initialization, preservation, exit consequence, and a decreasing termination measure for the prefix-count loop.
4. Insert an off-by-one or >= defect in a disposable copy; find a counterexample and explain which proof-to-code correspondence fails.

Coding slice: Work in `labs/student/lab_06_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A statement like "the count is correct" is too vague without saying which prefix, at what point in the loop, and under which health rule. Do not omit termination.

Independent transfer task

Change "positive health" to "health at least 3." Which parts of the proof change?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What are the two essential parts of induction?
2. What is 1+...+100?
3. Why does a mathematical proof not eliminate implementation tests?
4. Can a loop invariant be true while a loop never terminates?
5. State a precise invariant after processing i health values.

Assigned study

R06: induction, invariants, and state machines. Supply two independently written proofs in the module problem set; computed examples support but do not replace them.

Original reading assignment retained: R06: induction, invariants and proof methods.

Lesson 06.4 — Counting, graphs and recurrences

Learning outcome: Model combinations and recursive work without brute-force guessing.

Count choices without assuming independence of restrictions

If four equipment slots each allow three choices and choices impose no restrictions on one another, there are 3^4 = 81 loadouts. This is the product rule for combinatorial choices, not a probability assumption about how players select them. When a restriction couples slots, multiply only after conditioning on the relevant cases, or subtract a carefully counted forbidden set.

The sum rule adds counts of disjoint alternatives. Counting "red or heavy items" by adding all red and all heavy items double-counts items that are both. Inclusion-exclusion for two sets is |A union B| = |A| + |B| - |A intersection B|. Writing the overlap explicitly prevents an inflated estimate.

Model a graph before choosing its algorithm

A graph has vertices and edges. In a navigation graph, an edge might mean legal movement; in a build graph, it might mean one asset must precede another. Directed edges and undirected edges model different relationships. A simple undirected graph has no self-loop and at most one edge per unordered pair of vertices.

The count n(n-1)/2 follows because each of n vertices can pair with n-1 others, but that counts each undirected pair twice. Directed edges without self-loops have at most n(n-1) choices. These bounds are not interchangeable.

A startup order is a topological order

Let an edge A -> B mean A must finish before B begins. A topological order places every prerequisite before its dependent. A directed cycle prevents such an order: following its obligations eventually requires the first node to precede itself. Acyclic graphs can have multiple valid orders when some nodes are independent.

The reference routine repeatedly takes a zero-indegree node, outputs it, and removes its outgoing obligations. Remaining nodes with no zero-indegree choice reveal a cycle. It uses sorting to make tie-breaking reproducible, not because every topological-sort algorithm needs sorting. The supplied routine is an inspectable educational implementation; do not attribute the optimal O(V+E) bound to its repeatedly sorted Python lists.

A recurrence separates depth from total work

Define H(1)=0 and H(n)=1+H(floor(n/2)) for integer n>1. This counts floor-halvings and gives floor(log2(n)). With ceiling halves instead, the count is ceil(log2(n)). A recursive call chain's depth is not necessarily its total operation count.

For a single half-sized branch doing constant extra work, T(n)=T(n/2)+1 has logarithmic cost for powers of two. For two half-sized branches plus linear work, T(n)=2T(n/2)+n has n work at each of log2(n) levels, plus its leaves: the cost grows as n log n. The models have similar depth but very different totals.

Big-O states an upper-growth bound under a cost model. It is not a duration in milliseconds. A model may count comparisons while an actual program also spends time allocating, sorting, and accessing memory. Preserve both the symbolic argument and a measured experiment without treating either as the other. [R06]

Worked derivation

Use nodes assets, device, and scene, with assets -> scene and device -> scene. Either assets,device,scene or device,assets,scene is valid. Adding scene -> assets creates a cycle and eliminates every valid startup order.

For four three-choice slots, suppose the only forbidden situation is choosing option A in both slots 1 and 2. The forbidden count is 1*1*3*3 = 9, so 81-9 = 72 remain.

For floor-halving, 1000 becomes 500,250,125,62,31,15,7,3,1: nine divisions. For ceiling-halving it becomes 500,250,125,63,32,16,8,4,2,1: ten. Both must be distinguished from a real-valued logarithm.

Runnable demonstration

Run `python run_example.py 06.4` from the extracted package folder. Predict before executing.

```
from mathkit import halving_steps, topological_order
graph = {"assets": ["scene"], "device": ["scene"], "scene": []}
print("order:", topological_order(graph))
print("loadouts:", 3 ** 4)
print("complete graph edges:", 5 * 4 // 2)
print("halving steps:", halving_steps(1024))
```

Recorded expected output:

```
order: ['assets', 'device', 'scene']
loadouts: 81
complete graph edges: 10
halving steps: 10
```

Practical assignment

Original brief: Build a dependency graph for game startup and identify an invalid cycle. Count loadout combinations with and without a restriction. Pass: explain the recurrence for halving a search range and distinguish depth from total work.

1. Implement halving_steps(n) using repeated floor division, accepting positive integers only. Do not implement it by rounding a floating logarithm.
2. Draw a startup graph with at least six nodes, label edge meaning, give a valid order, then add an explicit invalid cycle.
3. Count the four-slot loadouts with and without the stated restriction and check by enumeration.
4. Derive both a one-branch halving recurrence and a two-branch divide-and-combine recurrence. Distinguish comparisons, depth, and elapsed time.

Coding slice: Work in `labs/student/lab_06_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not multiply counts across coupled choices, add overlapping alternatives without correction, or equate recursion depth with total work.

Independent transfer task

There are six nodes. Compare the maximum directed and undirected edge counts without self-loops.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. How many combinations arise from five independent binary choices?
2. How many edges can a simple undirected graph with five vertices have?
3. Why is a dependency cycle a problem for a simple startup order?
4. Four slots have three choices each; option A in both first slots is forbidden. How many loadouts remain?
5. Does the repeatedly sorted reference topological routine automatically have O(V+E) time?

Assigned study

R06: counting, directed graphs, recurrences, and asymptotic notation. Use a diagram or adjacency list with an explicit edge convention.

Original reading assignment retained: R06: counting, graphs, recurrences and asymptotic notation.